

操作系统校赛 · 内核实现赛道

PROCESS ProcMgrSim

—— 进程管理内核模拟器 ——

COW 写时复制

STRIDE 调度

管道 IPC

级联退出

5000+
LINES

40+
TESTS

6
DEMOS

02

实现了 Linux 内核的进程管理核心机制

COW 写时复制、Stride 公平调度、管道 IPC——三大核心机制，均基于 Linux 内核的真实设计模式，在用户态完整还原。

DESIGN PHILOSOPHY

ProcMgrSim 在用户态完整模拟了 Linux 内核的进程管理全链路：从 fork 创建进程、COW 写时复制内存、Stride 公平调度 CPU、到管道 IPC 进程间通信、级联退出与孤儿回收。所有实现均可交互操作、可视化验证。

DESIGN PHILOSOPHY

01 **用户态模拟**
不依赖硬件，零配置即可运行

02 **完整生命周期**
fork → 调度 → 通信 → waitpid → exit

03 **可视化验证**
进程树 · 甘特图 · 内存映射 · 上下文切换

03

NEXT

五个核心场景，覆盖进程管理全链路

01



FORK

创建进程

fork + ps + tree
观察进程树父子关系

02



COW

写时复制

mem + chart mapping
验证零拷贝内存共享

03



STRIDE

公平调度

nice + ps + chart stride
验证 CPU 时间比例

04



IPC

管道通信

pipe + write + read
父写子读，数据传递

05



DMESG

内核日志

dmesg 命令
回放所有内核操作记录

VS DEMO TIME

—— 切换到 VSCode 进行现场演示 ——

PRESS ESC → OVERVIEW → SELECT VSCODE WINDOW

04

创新点一 · COPY-ON-WRITE

COW 写时复制

kernel/memory_manager.py

```
def fork_page_table(self, parent_pt):
    """fork 时复制页表（零拷贝）"""
    child_pt = PageTable()
    for page, entry in parent_pt.entries:
        child_pt.map(page, entry.frame_id)
        entry.is_writable = False # 只读
        self._fm.incref(entry.frame_id)
    return child_pt

def on_page_write(self, pt, page_num):
    """写入时触发 COW"""
    entry = pt.get_entry(page_num)
    if entry.is_writable: return False
    # ref_count > 1 → 复制物理帧
    new_frame = self._fm.allocate()
    self._fm.copy_frame(entry.frame_id, new_frame)
    entry.frame_id = new_frame
    entry.is_writable = True
```

HOW IT WORKS

■ fork 前

父进程拥有独立页表
物理帧 ref_count = 1

■ fork 后

子进程页表指向同一物理帧
ref_count = 2 · 零拷贝 · 零额外内存

■ 写入后

触发 COW → 只复制被修改的那一帧
其余帧继续共享 · 节省内存

"fork 时零内存拷贝，只有写入时才复制。"

- LINUX KERNEL DESIGN PATTERN

05

COW 验证结果

01 TRADITIONAL FORK

全量复制页表

fork 时将父进程的全部页表项复制给子进程，每个物理帧的引用计数不变，子进程获得独立的内存副本。

```
// 实测数据 (4 页 × 4KB)
fork 前空闲帧: 252
fork 后空闲帧: 248
多分配 4 帧 · 额外开销 16KB
```

02 OURS · COW

写时复制共享

fork 时仅复制页表结构，父子进程的页表项指向同一物理帧，ref_count 递增。仅在写入时触发按需复制。

```
// 实测数据 (4 页 × 4KB)
fork 前空闲帧: 252
fork 后空闲帧: 252
零额外分配 · 内存开销 = 0
```

06

创新点二 · STRIDE

Stride 公平调度

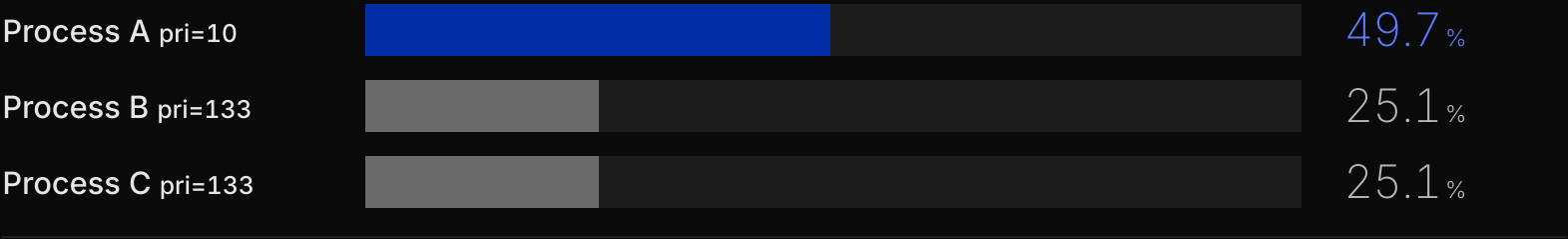
```
# kernel/scheduler.py
BIG_STRIDE = 1_000_000

def register(self, pid, priority=128):
    stride = 1_000_000 / (256 - priority)
    self._registry[pid] = {
        'stride': stride, 'pass_value': 0.0,
    }

def dequeue(self):
    return heapq.heappop(self._heap)

def update_after_run(self, pid, ticks):
    entry['pass_value'] += entry['stride'] * ticks
```

VERIFIED · 200 TICKS



- 数学证明
stride_A / stride_B = 8130/4065 = 2:1
CPU 时间比 = 1/stride_A : 1/stride_B = 2:1

Stride 调度的公平性不靠经验，靠数学。比例由优先级精确决定，可证明、可复现。

- LINUX CFS SCHEDULER MATHEMATICAL BASIS

07

管道进程间通信

```
# kernel/ipc.py
class Pipe:
    BUFFER_SIZE = 4096

    def write(self, data: bytes) -> int:
        """写入管道缓冲区"""
        for byte in data:
            self._buffer.append(byte)
        return len(data)

    def read(self, size: int) -> bytes:
        """从管道读取"""
        result = bytearray()
        while self._buffer and len(result) < size:
            result.append(self._buffer.popleft())
        return bytes(result)
```

EXECUTION FLOW

- 01 **pipe()**
创建管道 → read_fd=3, write_fd=4
- 02 **fork()**
子进程继承文件描述符表
- 03 **write(fd=4, "Hello")**
父进程写入 → 内核缓冲区
- 04 **read(fd=3) → "Hello"**
子进程读取 ← 内核缓冲区

// 文件描述符表
父进程: fd=3→read fd=4→write
子进程: fd=3→read fd=4→write (继承)

08

进程生命周期

六状态机 + 级联退出

STATE MACHINE

NEW → READY

READY ⇌ RUNNING

RUNNING → ZOMBIE

RUNNING → SLEEPING

SLEEPING → READY

ZOMBIE → TERMINATED

所有转换通过 `set_state()` 验证合法性

CASCADING EXIT

级联退出三阶段:

- 1. 进程状态 → ZOMBIE
- 2. 子进程成为孤儿
- 3. 孤儿重新挂到 init

`reparent_orphans()` 自动回收

FORK BOMB PROTECTION

安全机制:

- `max_children` = 64
- 最大进程数 = 64
- PID 单调递增
- init 不可杀死

与 Linux `ulimit` 原理一致

09

五层分层架构

L05	用户交互层 interactive.py
L04	可视化引擎 visualizer/
L03	系统调用层 syscall/syscall.py
L02	内核核心层 kernel/
L01	数据结构层 core/

IMPLEMENTATION DETAILS

进程管理器 · ProcessManager

PID 全局唯一分配 · 最大进程数限制(fork bomb防护)
create_process() 7步流程：检查上限→分配PID→创建PCB→注册进程树→注册调度器→分配页表→设READY

COW 内存管理 · COWManager

PageTableEntry: frame_id + is_valid + is_writable + is_dirty + ref_count
fork_page_table(): 复制PTE+设只读+incrcf · on_page_write(): COW触发+copy_frame

Stride 调度器 · StrideScheduler

最小堆 $O(\log n)$ 调度 · pass_value 溢出自动重置
 $stride = 1,000,000 / (256 - priority)$ · update_after_run() 累加步长

IPC 管道 · IPCManager

Pipe: 4KB环形缓冲区 · 阻塞读写 · fork时复制FD表
fd=0,1,2保留stdin/stdout/stderr · 管道从fd=3开始分配

10

项目数据

LINES OF CODE

5000+

纯 Python · 零外部依赖

TEST CASES

40+

6 个测试文件全覆盖

DEMO SCRIPTS

6

fork · COW · stride · pipe

SHELL COMMANDS

15+

完整交互式 Shell

MODULE BREAKDOWN

core/PCB · ProcessTree
ProcessState 状态机**kernel/**Kernel · ProcessManager
COWManager · StrideScheduler**syscall/**SyscallHandler
fork · exit · waitpid · pipe**visualizer/**charts · dashboard
tree_view · memory_view**tests/**6 文件 40+ 用例
PCB·树·调度·内存·内核

THREE INNOVATIONS

三个核心 创新点

用 Python 模拟 Linux 内核的进程管理机制，让学生直观理解操作系统原理。

01 COW 写时复制

fork 时零内存拷贝，引用计数 + 脏页追踪 + 按需复制。与 Linux 内核 `copy_on_write()` 设计一致。

02 Stride 公平调度

基于 Linux CFS 的数学基础， $O(\log n)$ 最小堆调度，优先级比例精确可控，`pass_value` 溢出自动重置。

03 完整进程生命周期

六状态机 + 级联退出 + 孤儿回收 + 僵尸清理 + fork bomb 防护。覆盖进程管理全链路。

12

后续扩展方向

01



缺页中断

实现缺页中断机制

让 COW 更真实：访问未映射页时触发中断，按需分配物理帧，支持页面置换算法。

02



多级页表

实现多级页表

二级页表 + TLB 快表，更接近真实内核的内存管理方式。

03



更多 IPC

扩展 IPC 机制

共享内存、消息队列、信号量。支持更复杂的进程间协作模式。

Thanks!

☰ 谢谢！

感谢各位老师的聆听
欢迎指导与交流

🔗 gitee.com/zjh3432512933/process-management

```
# proc_mgr_sim output
def main():
    kernel = Kernel(max_processes=64)
    shell = OShell(kernel)
    shell.run()
```

```
# output:
print("谢谢！")

谢谢！
```